Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА 5
по дисциплине:
«Управление конфигурацией и сопровождение программного обеспечения»

на тему:
«Автоматические проверки перед выпуском Развёртывание и возврат версии»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Гуцол Д.В.

Оценка:_____
Дата: 22.10.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

Цель работы: на учебном стенде GitOps обнаружить неверную конфигурацию, исправить её, пройти автоматические проверки, выпустить архив конкретного коммита, затем развернуть версии, зафиксировать инцидент после обновления, вернуть известный рабочий выпуск и подтвердить сохранность истории Git.

Пара 1. Автоматические проверки перед выпуском

Рабочая папка стенда: git-ops-p1. Все команды выполнялись в Git Bash из корня стенда, без перехода в каталог work. Имя автора коммитов: Trifonov D.S.

A1. Подготовка стенда

Выполнены pwd, bash env.sh и bash setup.sh. Создан репозиторий work на ветке main. env.sh подтвердил наличие git, bash, tar, sha256sum и mktemp. Включены расширения имён файлов. Просмотрены work/config.txt, README.txt, check.sh и tests.sh без изменений.

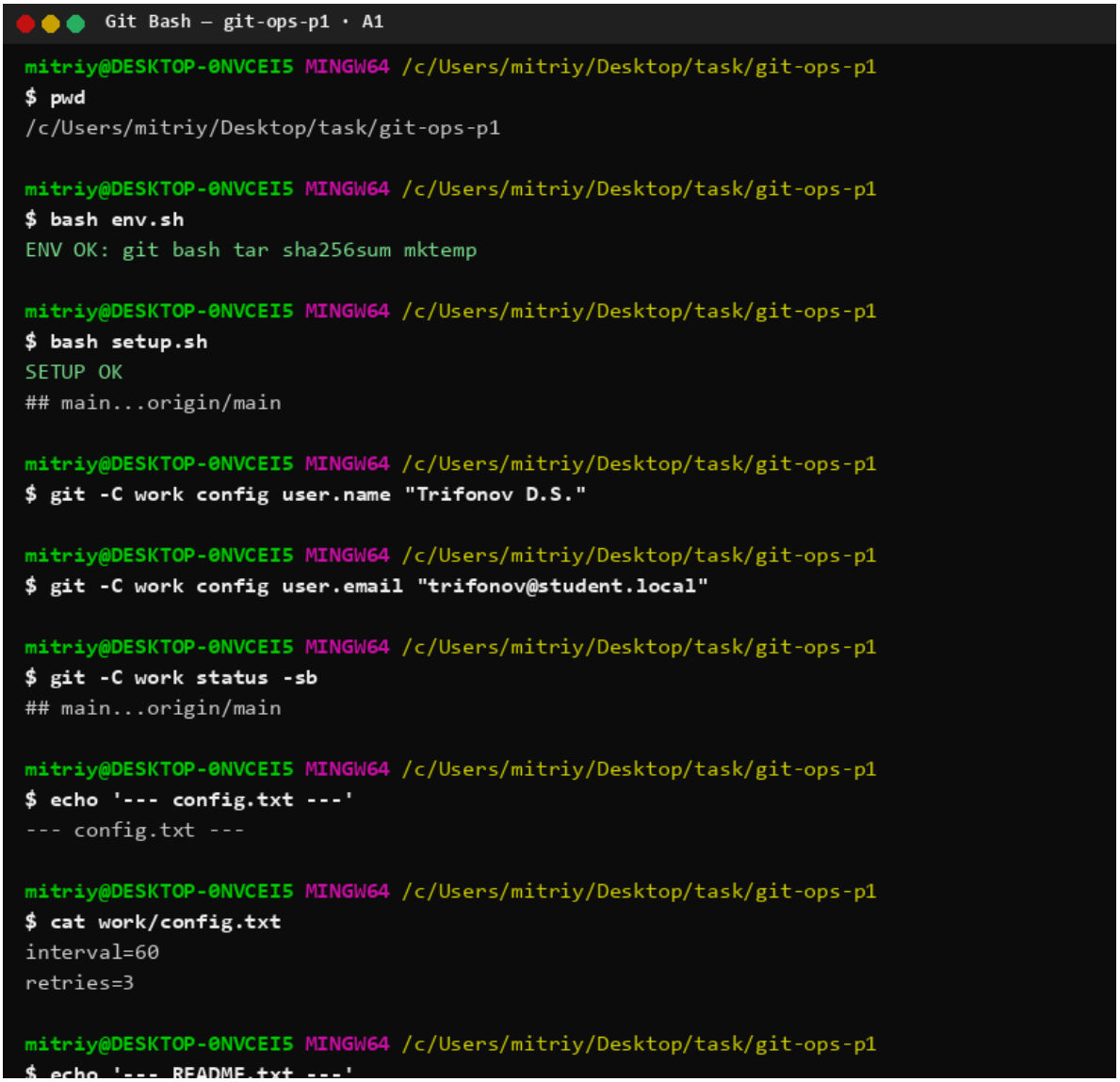


Рисунок 1 – Подготовка стенда git-ops-p1: env.sh, setup.sh, статус main

Роль файлов. config.txt задаёт interval и retries; допустимы только interval=30 или 60 при retries=3. README.txt описывает выбранный интервал и должен совпадать с config.txt. check.sh — производственная проверка выпуска (PASS/FAIL). tests.sh содержит четыре проверки самого check.sh: два позитивных сценария (30 и 60) и два ожидаемых отказа (недопустимый interval=0 и расхождение README с config).

Блокнот — work/check.sh

```
#!/usr/bin/env bash
set -euo pipefail
cfg="${1:-}"
readme="${2:-}"
if [[ -z "$cfg" || -z "$readme" ]]; then
  echo "Usage: check.sh CONFIG README" >&2
  exit 2
fi
interval="$(grep -E '^interval=' "$cfg" | tail -n1 | cut -d= -f2 | tr -d '\r')"
retries="$(grep -E '^retries=' "$cfg" | tail -n1 | cut -d= -f2 | tr -d '\r')"
if [[ "$interval" != "30" && "$interval" != "60" ]] || [[ "$retries" != "3" ]]; then
  echo "FAIL: config must contain interval=30 or 60 and retries=3"
  exit 1
fi
if ! grep -q "$interval" "$readme"; then
  echo "FAIL: README and config disagree"
  exit 1
fi
echo "PASS for $interval"
```

Рисунок 2 – Фрагмент check.sh: допустимые interval=30 или 60 и retries=3

Блокнот — work/tests.sh

```
#!/usr/bin/env bash
set -euo pipefail
ROOT="$(cd "$(dirname "$0")" && pwd)"
CHECK="$ROOT/check.sh"
tmp="$(mktemp -d)"
cleanup() { rm -rf "$tmp"; }
trap cleanup EXIT

pass() { echo "TEST PASS: $1"; }

# Test 1: valid interval=30 and matching README must pass.
printf 'interval=30\nretries=3\n' > "$tmp/c1"
printf 'Service configuration\nMonitoring interval: 30 seconds.\n' > "$tmp/r1"
if "$CHECK" "$tmp/c1" "$tmp/r1" >/dev/null; then
  pass "valid interval 30 accepted"
else
  echo "TEST FAIL: valid interval 30 was rejected"
  exit 1
fi

# Test 2: valid interval=60 and matching README must pass.
printf 'interval=60\nretries=3\n' > "$tmp/c2"
printf 'Service configuration\nMonitoring interval: 60 seconds.\n' > "$tmp/r2"
if "$CHECK" "$tmp/c2" "$tmp/r2" >/dev/null; then
  pass "valid interval 60 accepted"
else
  echo "TEST FAIL: valid interval 60 was rejected"
  exit 1
```

Рисунок 3 – Фрагмент tests.sh: тесты 1–4, включая ожидаемый отказ проверки

A2. Неверное изменение и отказ проверки

В config.txt записано interval=0 при retries=3. Сразу после bash ci.sh verify сохранён код echo $?. Получен ожидаемый отказ: код 1, выпуск не создан. Сообщение: FAIL: config must contain interval=30 or 60 and retries=3. Протоколы

сохранены в evidence/01_failed.txt и evidence/01_exit.txt. Выпуск должен остановиться, чтобы недопустимая конфигурация не попала в архив коммита.

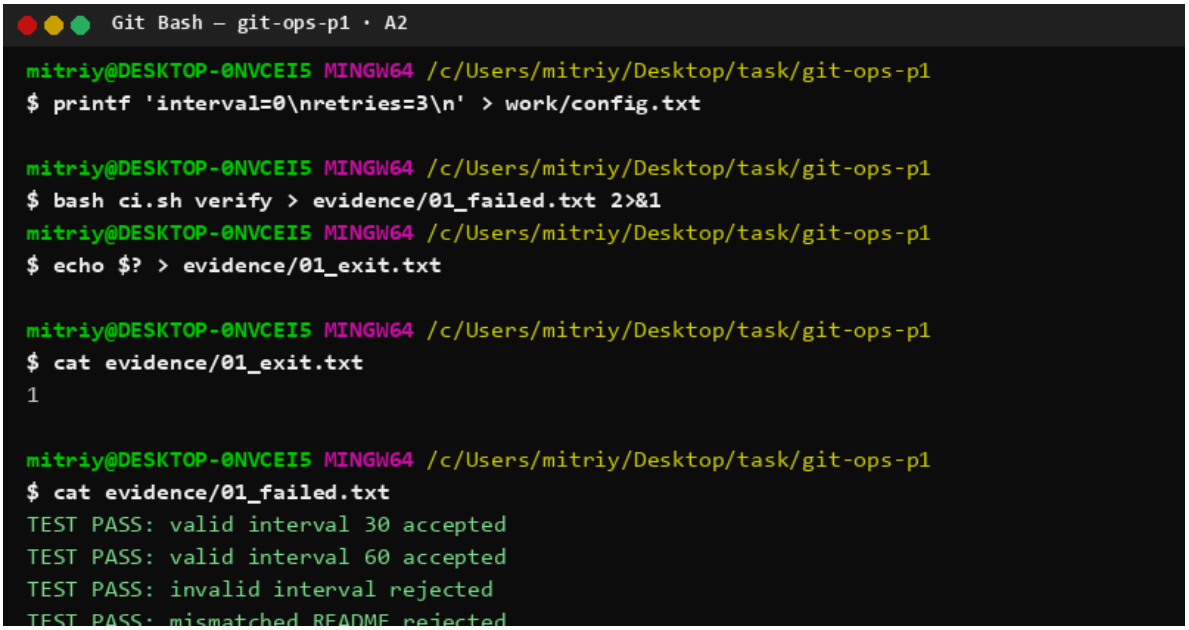


Рисунок 4 – Ожидаемый отказ ci.sh verify: код 1 и сообщение FAIL

A3. Исправление конфигурации и описания

Установлены interval=30, retries=3 и README с текстом «Monitoring interval: 30 seconds.». diff показал изменение только config.txt и README.txt. ci.sh verify вернул PASS for 30, четыре TEST PASS и VERIFY PASS.

Тест 1 принимает эталон interval=30 с согласованным README. Тест 2 принимает эталон interval=60. Тест 3 успешен, если check.sh отвергает interval=0. Тест 4 успешен, если check.sh отвергает рассинхрон README и config. Для тестов 3 и 4 успех — это ожидаемый отказ проверки, а не PASS самого check.sh.

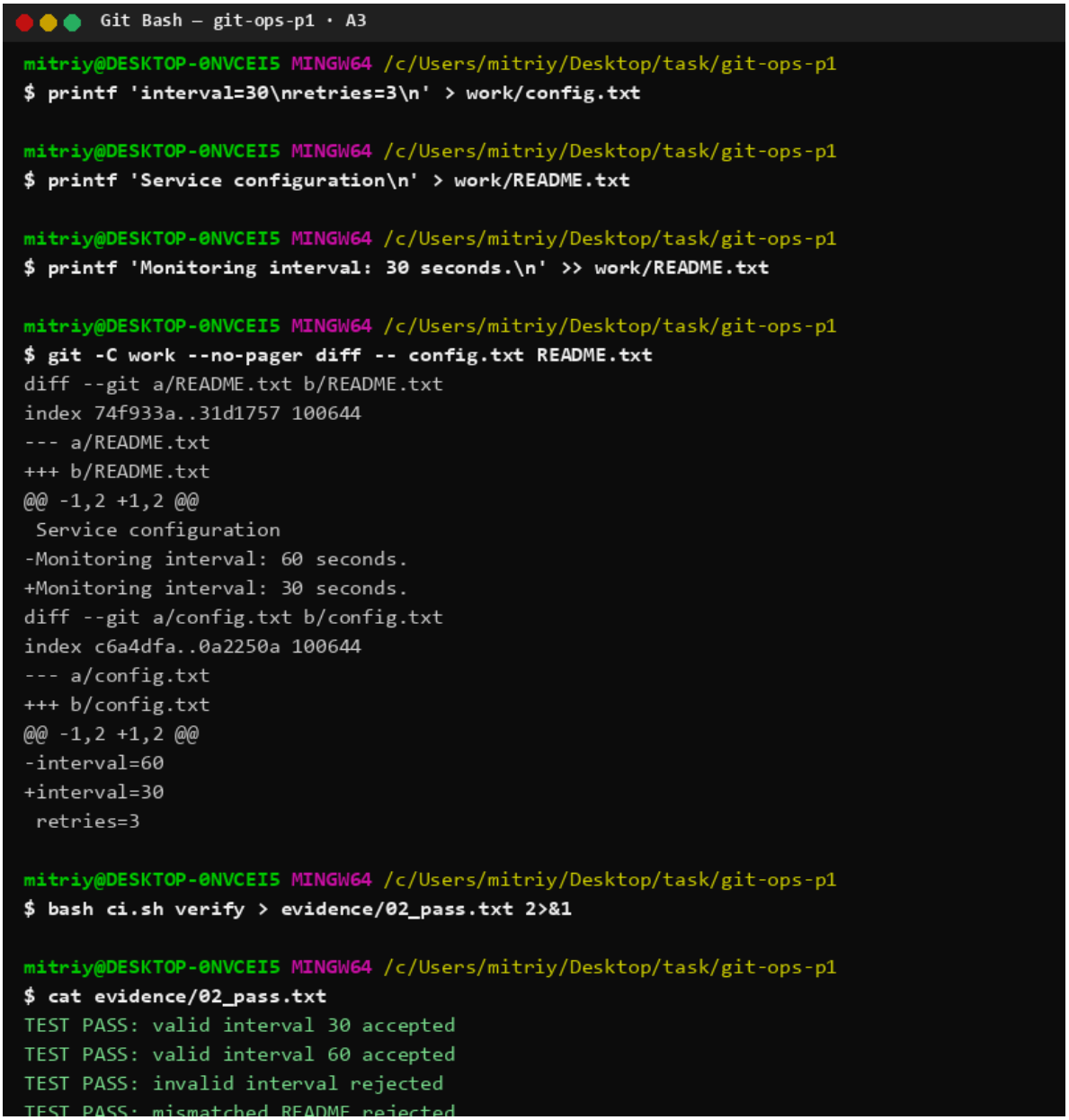


Рисунок 5 – Исправление config/README и VERIFY PASS с четырьмя TEST PASS



Рисунок 6 – Исправленный work/config.txt (interval=30)



Рисунок 7 – Исправленный work/README.txt, согласованный с interval=30

A4. Сохранение проверенной версии

Файлы config.txt и README.txt добавлены и сохранены коммитом «Set verified monitoring interval to 30». Рабочее дерево чистое, ветка main впереди origin на 1 коммит. В истории два коммита. bash ci.sh release завершился RELEASE PASS. В artifacts три файла: release.tar, release.sha256, manifest.txt. Хеш в manifest совпадает с git rev-parse HEAD.

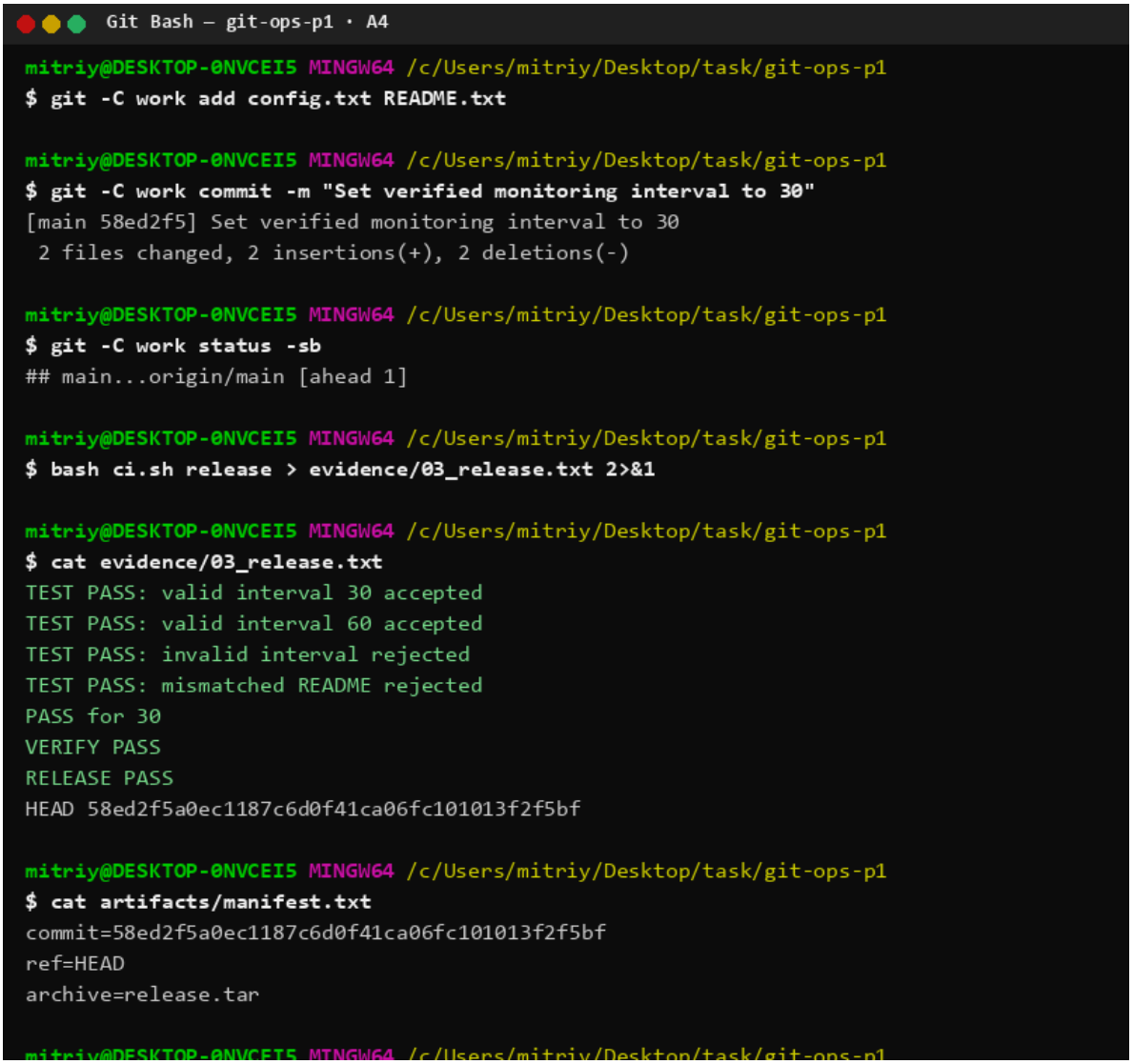


Рисунок 8 – Коммит проверенных изменений, RELEASE PASS и manifest на HEAD

A5. Проверка выпуска

sha256sum -c release.sha256 вернул release.tar: OK — контрольная сумма архива совпадает с записанной при выпуске. Это подтверждает целостность файла, но не содержимую корректность сервиса, не нагрузку и не полноту истории. Распаковка в inspect и bash inspect/check.sh дали PASS for 30. В архиве есть config.txt, README.txt, check.sh, tests.sh и нет каталога .git, потому что выпуск — снимок файлов версии, а не клон репозитория.

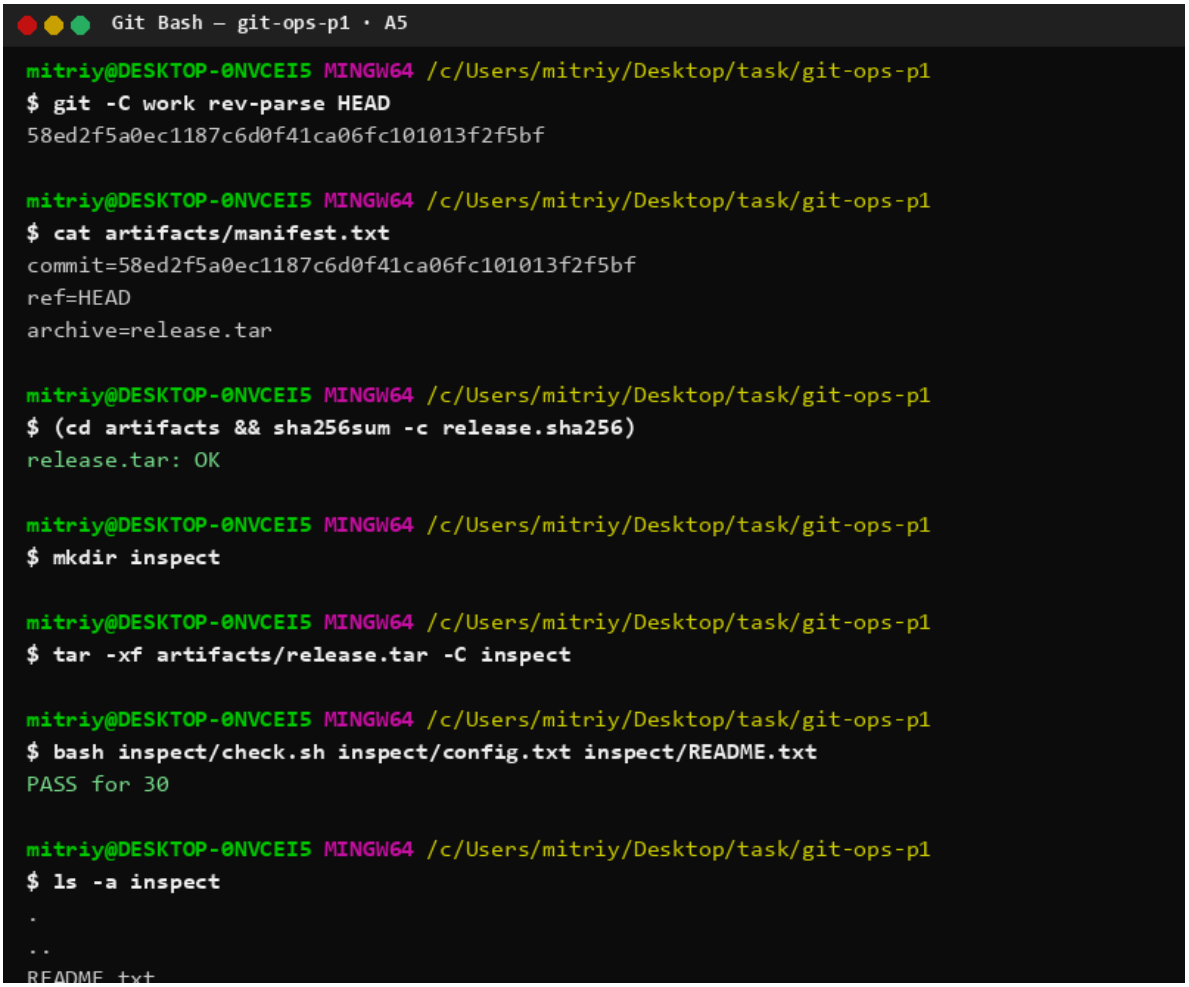


Рисунок 9 – SHA256 OK, PASS for 30 в inspect, отсутствие .git

A6. Отчёт и сдача пары 1

REPORT.txt заполнен без пометок [заполнить] и сохранён в UTF-8. bash pack.sh вернул READY. В submission есть report.txt, work.bundle, history.txt, evidence, release.tar, release.sha256 и manifest.txt. Содержимое submission упаковано в GitOps_pair1_result.zip рядом с каталогом сдачи.

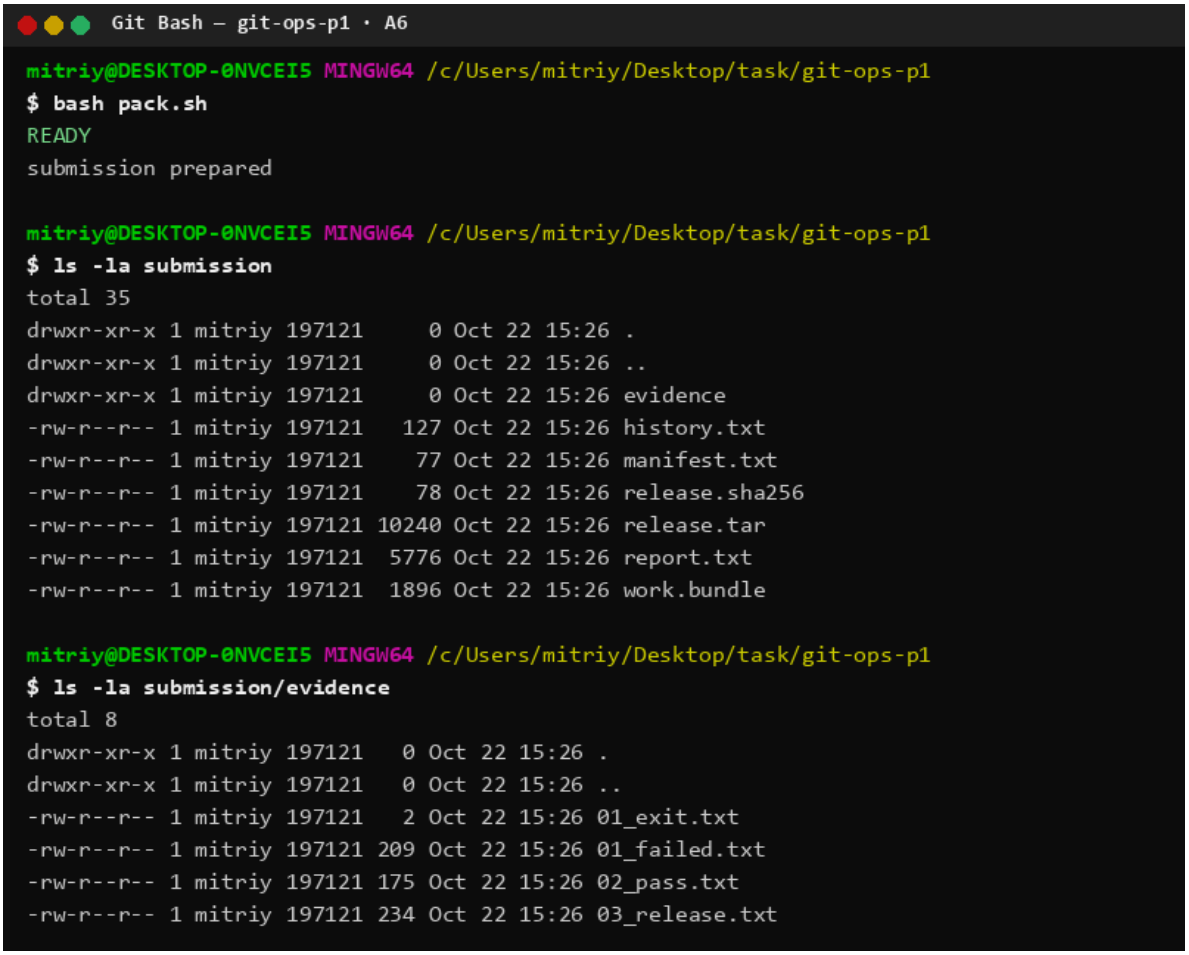


Рисунок 10 – pack.sh: READY и состав submission пары 1

Блокнот — REPORT.txt

```
Практикум пары 1. Автоматические проверки перед выпуском
Студент: Трифонов Д.С.
Группа: 12-25РПм
Дата: 22.10.2026

A1. Роль файлов work/config.txt, README.txt, check.sh и tests.sh
config.txt задаёт рабочие параметры сервиса: interval (период опроса) и retries (число повторов). Допустимы то
README.txt описывает выбранную конфигурацию человекочитаемо; число секунд в тексте должно совпадать с interval
check.sh — производственная проверка выпуска: читает два файла и возвращает PASS for 30/60 либо FAIL. Именно е
tests.sh — учебный набор из четырёх проверок самого check.sh. Он не выпускает продукт, а подтверждает, что пра

A2. Почему выпуск должен остановиться. Код возврата и точное сообщение
После записи interval=0 проверка обязана отказать: значение 0 не входит в допустимый набор 30 или 60. Выпуск н
Код возврата: 1
Точное сообщение: FAIL: config must contain interval=30 or 60 and retries=3
Протокол сохранён в evidence/01_failed.txt и evidence/01_exit.txt.

A3. Объяснение четырёх тестов
Тест 1: эталонные файлы с interval=30 и согласованным README должны получить PASS. Это прямой позитивный сцена
Тест 2: то же для interval=60 — второе допустимое значение тоже принимается.
Тест 3: interval=0 должен быть отвергнут. Успех теста — это ожидаемый отказ check.sh; если бы check пропустил
Тест 4: при interval=30 и README с числом 60 check.sh должен отказать из-за расхождения описания и конфигураци
На рабочей копии после исправления получены PASS for 30, четыре TEST PASS и VERIFY PASS.

A4. Сохранённые файлы, хеш в manifest и связь с историей
В коммит вошли только config.txt и README.txt — те же файлы, что правились в A3. Сообщение коммита: Set verifi
В истории два коммита: исходный baseline и проверенное исправление.
manifest.txt:
commit=58ed2f5a0ec1187c6d0f41ca06fc101013f2f5bf
ref=HEAD
archive=release.tar
Хеш commit в манифесте совпадает с git rev-parse HEAD (58ed2f5a0ec1187c6d0f41ca06fc101013f2f5bf), то есть архи

A5. Что подтверждает SHA256 OK. Ограничения проверки. Почему в inspect нет .git
SHA256 OK подтверждает только целостность файла artifacts/release.tar относительно контрольной суммы, записанн
Ограничения: check.sh смотрит лишь interval/retries и наличие числа в README; не измеряется нагрузка, не запус
В inspect нет каталога .git, потому что выпуск — это tar с файлами версии (config.txt, README.txt, check.sh, t

Границы автоматических проверок
Автоматические проверки ловят формально недопустимую конфигурацию и рассинхрон README, но не доказывают коррек

Незавершённые этапы и ошибки
Все этапы A1–A6 выполнены. Отказ в A2 был ожидаемым и сохранён без ручной правки протоколов.
```

Рисунок 11 – Заполненный REPORT.txt пары 1

Git Bash — содержимое GitOps_pair1_result.zip

```
mitriy@DESKTOP-0NVCEI5 MINGW64 /c/Users/mitriy/Desktop/task/git-ops-p1
$ unzip -l GitOps_pair1_result.zip
history.txt
manifest.txt
release.sha256
release.tar
report.txt
work.bundle
evidence/01_exit.txt
evidence/01_failed.txt
evidence/02_pass.txt
evidence/03_release.txt

файлов в архиве: 10
mitriy@DESKTOP-0NVCEI5
```

Рисунок 12 – Состав архива GitOps_pair1_result.zip

Пара 2. Развёртывание и возврат версии

Рабочая папка стенда: git-ops-p2. Использован отдельный проект второй пары, без переноса work из первой.

Б1. Подготовка двух версий

После setup в истории два коммита и два тега; main указывает на v1.1.0. По diff v1.0.0 → v1.1.0 изменены две строки: interval=60 на interval=30 и «Monitoring interval: 60 seconds.» на «Monitoring interval: 30 seconds.». Прогноз: условию среды «interval не меньше 60» удовлетворяет только v1.0.0. HEAD до экспериментов сохранён в evidence/head_before.txt.



Рисунок 13 – Два тега, diff конфигурации и сохранённый HEAD

Б2. Первое развёртывание

bash deploy.sh v1.0.0: DEPLOYED v1.0.0, CURRENT: deployments/run_1. health.sh: HEALTH PASS: interval=60 meets this environment minimum of 60. commit в deployments/run_1/deployed.txt совпадает с git rev-parse v1.0.0. Каталог deployments моделирует установленные файлы; реальный сервис не запускался.

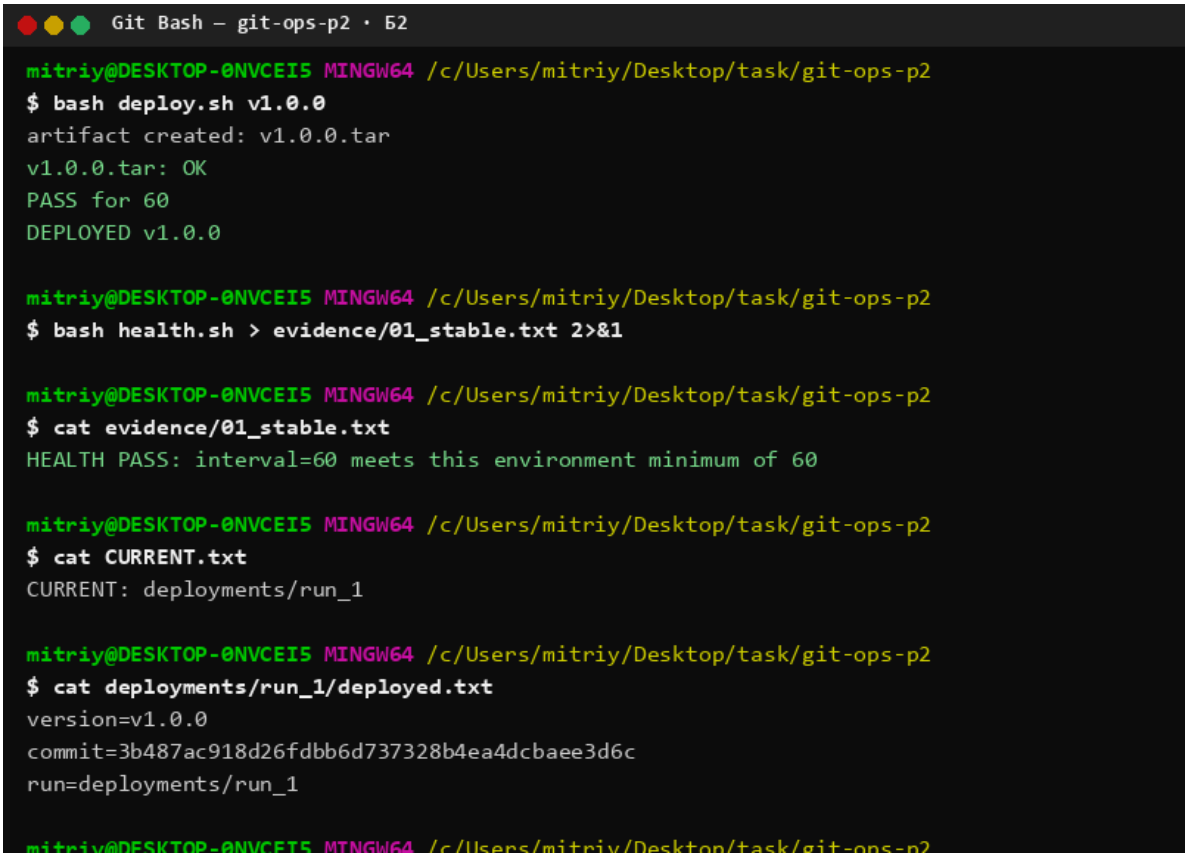


Рисунок 14 – Развёртывание v1.0.0 и HEALTH PASS

Б3. Обновление и инцидент

При deploy v1.1.0 базовая проверка архива проходит (PASS for 30), потому что check.sh допускает 30 и 60. health.sh проверяет другое правило — минимум среды 60 — и возвращает HEALTH FAIL для interval=30, код 1. CURRENT: deployments/run_2. Протокол 01_stable.txt не перезаписывался. Нагрузка не измерялась; зафиксирован только отказ проверки минимума interval.

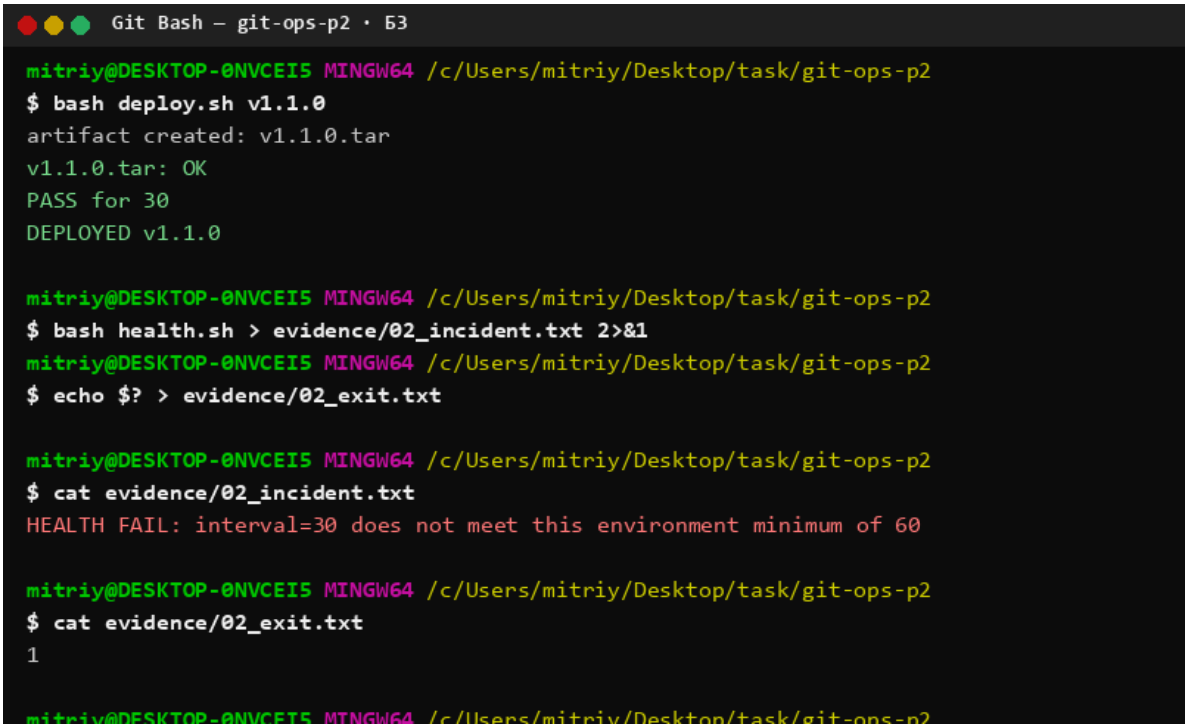


Рисунок 15 – Инцидент v1.1.0: HEALTH FAIL, код 1, run_2

Б4. Возврат к известной рабочей версии

Повторный deploy.sh v1.0.0 использовал уже существующий artifacts/v1.0.0.tar (сообщение reusing artifact) и снова проверил SHA256 (v1.0.0.tar: OK). Файлы возврата взяты из этого архива, а не из переписывания Git. health выполняется снова, потому что успешная распаковка не равна готовности среды. run_3 — порядковый номер третьего развёртывания, не «версия 3». CURRENT: deployments/run_3, версия v1.0.0, HEALTH PASS.

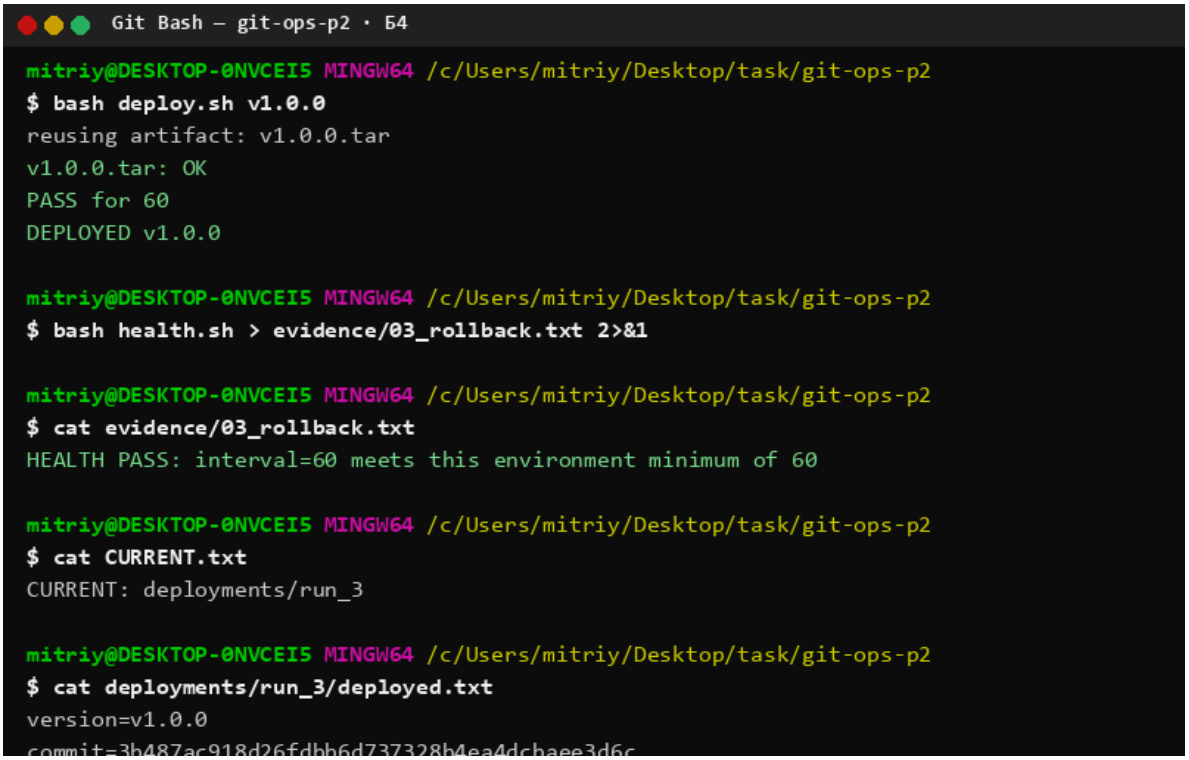


Рисунок 16 – Откат на архив v1.0.0, CURRENT run_3, HEALTH PASS

Б5. Сохранность истории и журнал

HEAD до и после совпадает; main по-прежнему на v1.1.0; теги v1.0.0 и v1.1.0 сохранены. Журнал evidence/deployments.tsv содержит порядок v1.0.0, v1.1.0, v1.0.0. Рабочее дерево чистое. Возврат установленной версии не перемещает ветку и не удаляет коммиты.

Карточка инцидента. Симптом: после обновления на v1.1.0 health вернул код 1 для interval=30, активен run_2. Причина: контракт выпуска допускает 30, контракт среды требует минимум 60. Действие: повторно развёрнут архив v1.0.0 с проверкой SHA256 в run_3. Результат: HEALTH PASS, HEAD неизменен, оба тега на месте. Профилактика: не переключать CURRENT на выпуск, который не проходит минимум площадки. Дополнительная проверка перед переключением среды: прогон health (interval >= 60) по содержимому будущего каталога, а не только check.sh.

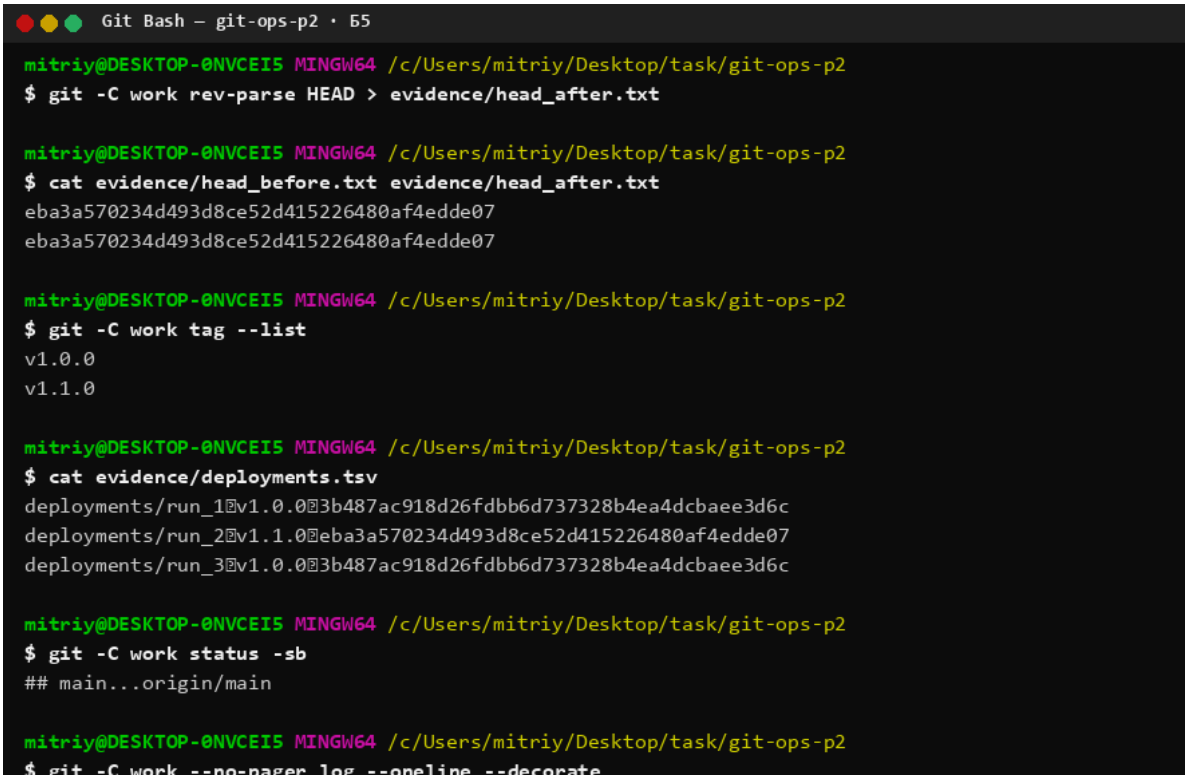


Рисунок 17 – Неизменный HEAD, оба тега и журнал трёх развёртываний

Б6. Отчёт и сдача пары 2

REPORT.txt заполнен, pack.sh вернул READY. В submission есть report.txt, work.bundle, evidence, архивы обеих версий, CURRENT.txt и active_version с файлами восстановленной v1.0.0. Содержимое упаковано в GitOps_pair2_result.zip.

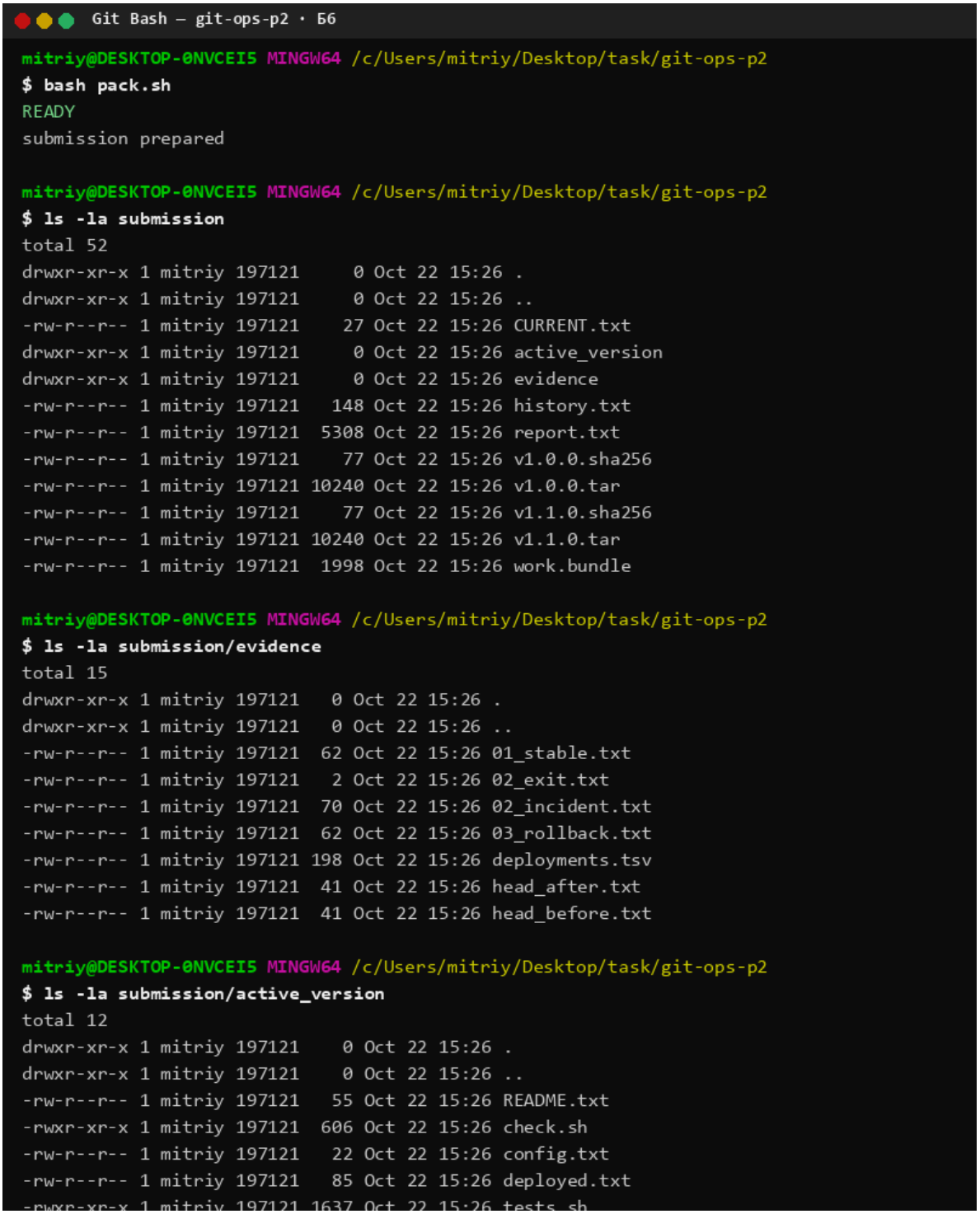


Рисунок 18 – pack.sh: READY и состав submission пары 2

Блокнот — REPORT.txt

```
Практикум пары 2. Развёртывание и возврат версии
Студент: Трифонов Д.С.
Группа: 12-25РПм
Дата: 22.10.2026

Б1. Изменённые строки по diff и прогноз: какая версия удовлетворит условию interval не меньше 60
По diff v1.0.0 → v1.1.0 изменены две строки:
- config.txt: interval=60 заменён на interval=30
- README.txt: Monitoring interval: 60 seconds. заменён на Monitoring interval: 30 seconds.
Прогноз: условию среды «interval не меньше 60» удовлетворяет только v1.0.0. Версия v1.1.0 пройдёт базовую прове

Б2. Исходное состояние. Сравнение deployed.txt и тега v1.0.0
Первое развёртывание: DEPLOYED v1.0.0, CURRENT: deployments/run_1, HEALTH PASS: interval=60 meets this environ
deployed.txt run_1:
version=v1.0.0
commit=3b487ac918d26fdbb6d737328b4ea4dcbaee3d6c
run=deployments/run_1
Поле commit совпадает с git rev-parse v1.0.0. Каталог deployments моделирует установленные файлы, реальный сер

Б3. Почему PASS при deploy и HEALTH FAIL совместимы. Симптом инцидента
Deploy проверяет учебный контракт выпуска: interval=30 или 60 и согласованный README. Для v1.1.0 это верно, по
health.sh проверяет другое правило — минимум конкретной среды (60). interval=30 этому правилу не удовлетворяет
Симптом: HEALTH FAIL: interval=30 does not meet this environment minimum of 60, код 1, CURRENT: deployments/ru
Нагрузка не измерялась; зафиксирован только отказ проверки минимума interval. Исходный успешный протокол 01_st

Б4. Откуда взяты файлы возврата и зачем снова нужен health. Смысл run_3
Файлы возврата взяты из уже существующего artifacts/v1.0.0.tar; deploy повторно проверил SHA256 и распаковал а
health нужен снова, потому что успешная распаковка не равна готовности среды: необходимо подтвердить, что акти
run_3 — это порядковый номер третьего развёртывания модели, а не «версия 3». CURRENT: deployments/run_3.
deployed.txt run_3:
version=v1.0.0
commit=3b487ac918d26fdbb6d737328b4ea4dcbaee3d6c
run=deployments/run_3

Б5. Карточка инцидента и дополнительная проверка перед переключением среды
Симптом: после обновления на v1.1.0 health вернул код 1 и сообщение HEALTH FAIL для interval=30; активным стал
Причина: правило среды требует interval не меньше 60, а новая версия несёт interval=30. Проверка выпуска и про
Действие: повторно развёрнут известный рабочий архив v1.0.0 (SHA256 проверен) в deployments/run_3.
Подтверждённый результат: HEALTH PASS для interval=60; HEAD до и после совпадает (eba3a570234d493d8ce52d41522
deployments/run_1␣v1.0.0␣3b487ac918d26fdbb6d737328b4ea4dcbaee3d6c
deployments/run_2␣v1.1.0␣eba3a570234d493d8ce52d415226480af4edde07
deployments/run_3␣v1.0.0␣3b487ac918d26fdbb6d737328b4ea4dcbaee3d6c
Рабочее дерево чистое.
Профилактика: не переключать среду на выпуск, который не удовлетворяет минимуму данной площадки, даже если ci/
Дополнительная проверка: перед сменой CURRENT выполнять health (или аналог «interval >= минимума среды») по со

Незавершённые этапы и ошибки
Все этапы Б1–Б6 выполнены. Отказ health в Б3 был запланированным инцидентом. reset --hard не применялся.
```

Рисунок 19 – Заполненный REPORT.txt пары 2

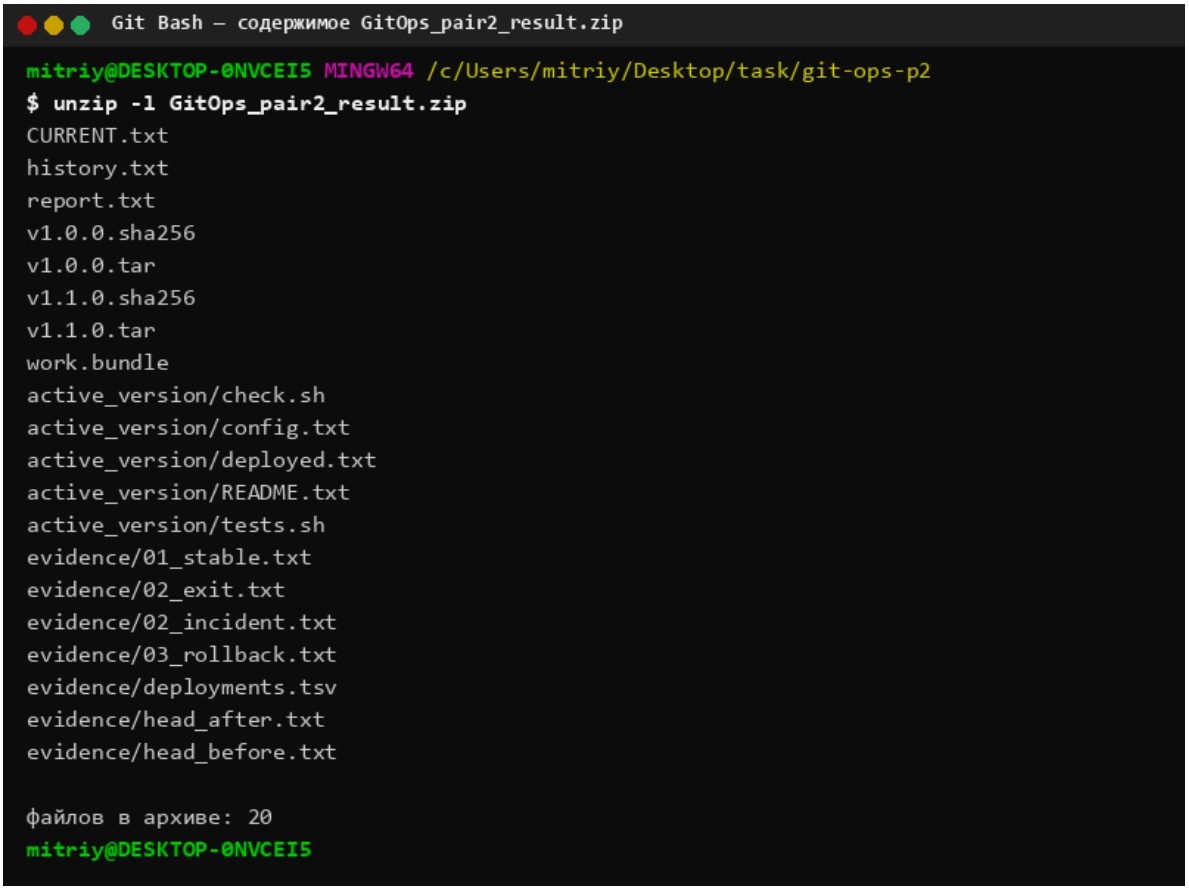


Рисунок 20 – Состав архива GitOps_pair2_result.zip

Выводы

1. Автоматическая проверка выпуска останавливает публикацию недопустимой конфигурации (interval=0, код 1) и пропускает согласованные значения 30/60 только после совпадения README и config.

2. Успешный выпуск привязан к конкретному коммиту: manifest.commit совпадает с HEAD, архив не содержит .git, целостность файла подтверждается SHA256.

3. Проверка выпуска и проверка среды — разные контракты: v1.1.0 проходит deploy/check, но даёт HEALTH FAIL при минимуме interval=60. Откат выполняется повторной установкой известного архива, без reset --hard.

4. История Git после инцидента сохранена: HEAD не изменился, теги на месте, журнал фиксирует три развёртывания. Подготовлены архивы сдачи GitOps_pair1_result.zip и GitOps_pair2_result.zip.